

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Database Management Systems

COURSE CODE: CSA0509

CO5: *Implement database operations using query processing, optimization, and transaction management to ensure consistency and reliability.* (BL3, BL4)

Activity Tool : Pseudocode Writing Task (Algorithm Design) (Medium)

Assessment Tool Weightage: 35%

Dr

QUESTIONS			Total Marks: 50
Q.No	Question	Bloom's Level	Marks
1	Write pseudocode for the complete query processing pipeline: from SQL input through parsing, optimization, and execution to result output.	BL3 – Apply	5
2	Design a pseudocode algorithm for cost-based query optimization that estimates and compares I/O costs for different join orderings.	BL4 – Analyze	5
3	Write pseudocode for the Nested Loop Join algorithm and analyze its time complexity in terms of buffer pages and tuple counts.	BL3 – Apply	5
4	Write a pseudocode algorithm to detect deadlocks in a transaction system using a Wait-For Graph (WFG) cycle detection.	BL4 – Analyze	5
5	Design a pseudocode for the Basic Two-Phase Locking protocol including lock request, grant, wait, and release phases.	BL3 – Apply	5
6	Write pseudocode for timestamp-based concurrency control using Thomas' Write Rule for handling conflicting read/write operations.	BL4 – Analyze	5
7	Write a pseudocode algorithm for the ARIES (Algorithm for Recovery and Isolation Exploiting Semantics) recovery technique with Analysis, Redo, and Undo phases.	BL3 – Apply	5

8	Design pseudocode for Shadow Paging recovery: maintain current and shadow page tables and roll back on failure.	BL3 – Apply	5
9	Write a pseudocode for Immediate Update recovery using UNDO/REDO log processing after a system crash.	BL3 – Apply	5
10	Write pseudocode for a database connectivity manager that handles connection pooling, query execution, and exception rollback.	BL4 – Analyze	5

Code of Conduct:

*I, Judy Allen J (192520055) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:
Judy Allen J

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient

Q1. Write pseudocode for the complete query processing pipeline: from SQL input through parsing, optimization, and execution to result output.

Answer:

Algorithm: Complete Query Processing Pipeline

BEGIN

INPUT SQL_Query

FUNCTION Query_Processing(SQL_Query)

 // Step 1: Lexical and Syntax Analysis

 Tokens ← TOKENIZE(SQL_Query)

 IF Tokens are INVALID THEN

 DISPLAY "Lexical Error"

 RETURN

 END IF

 Parse_Tree ← PARSE(Tokens)

 IF Parse_Tree contains syntax error THEN

 DISPLAY "SQL Syntax Error"

 RETURN

 END IF

 // Step 2: Semantic Validation

 IF referenced tables do not exist THEN

 DISPLAY "Table Not Found"

 RETURN

 END IF

 IF referenced columns do not exist THEN

 DISPLAY "Column Not Found"

 RETURN

 END IF

 IF user does not have required permission THEN

 DISPLAY "Access Denied"

 RETURN

 END IF

 // Step 3: Translate SQL into Logical Query Plan

 Logical_Plan ← CREATE_LOGICAL_PLAN(Parse_Tree)

 // Step 4: Query Rewriting

 Logical_Plan ← APPLY_SELECTION_PUSHDOWN(Logical_Plan)

 Logical_Plan ← APPLY_PROJECTION_PUSHDOWN(Logical_Plan)

 Logical_Plan ← SIMPLIFY_EXPRESSIONS(Logical_Plan)

```

// Step 5: Generate Candidate Physical Plans
Candidate_Plans ← GENERATE_EXECUTION_PLANS(Logical_Plan)

Best_Cost ← INFINITY
Best_Plan ← NULL

// Step 6: Cost-Based Optimization
FOR each Plan IN Candidate_Plans DO

    Cost ← ESTIMATE_COST(Plan)

    IF Cost < Best_Cost THEN
        Best_Cost ← Cost
        Best_Plan ← Plan
    END IF

END FOR

// Step 7: Execute Optimized Plan
Result ← EXECUTE(Best_Plan)

// Step 8: Return Output
DISPLAY Result

END FUNCTION

CALL Query_Processing(SQL_Query)

END

```

Explanation:

The query processor first checks the SQL statement for lexical, syntax, semantic, and authorization errors. It then converts the SQL statement into a logical query plan. The optimizer applies techniques such as selection and projection pushdown and generates multiple physical execution plans. The estimated cost of each plan is calculated, and the least-cost plan is selected for execution. Finally, the DBMS executes the optimized plan and returns the result.

Q2. Design a pseudocode algorithm for cost-based query optimization that estimates and compares I/O costs for different join orderings.

Answer:

Algorithm: Cost-Based Query Optimization

BEGIN

INPUT Relations R1, R2, ..., Rn

INPUT Available_Buffer_Pages

FUNCTION Estimate_Join_Cost(Relation1, Relation2)

Pages1 $\leftarrow$ NUMBER_OF_PAGES(Relation1)

Pages2 $\leftarrow$ NUMBER_OF_PAGES(Relation2)

// Block Nested Loop Join Cost

IF Available_Buffer_Pages > 2 THEN

Cost $\leftarrow$ Pages1 +
CEIL(Pages1 / (Available_Buffer_Pages - 2))
 $\times$ Pages2

ELSE

Cost $\leftarrow$ INFINITY

END IF

RETURN Cost

END FUNCTION

FUNCTION Find_Best_Join_Order(Relations)

Join_Orders $\leftarrow$ GENERATE_ALL_JOIN_ORDERS(Relations)

Minimum_Cost $\leftarrow$ INFINITY

Best_Order $\leftarrow$ NULL

FOR each Order IN Join_Orders DO

Total_Cost $\leftarrow$ 0

Intermediate_Result $\leftarrow$ Order[1]

FOR i $\leftarrow$ 2 TO LENGTH(Order) DO

Current_Relation $\leftarrow$ Order[i]

Join_Cost $\leftarrow$ Estimate_Join_Cost(
Intermediate_Result,
Current_Relation)

```

    Total_Cost ← Total_Cost + Join_Cost

    Intermediate_Result ←
        ESTIMATE_JOIN_RESULT(
            Intermediate_Result,
            Current_Relation)

END FOR

DISPLAY Order, Total_Cost

IF Total_Cost < Minimum_Cost THEN
    Minimum_Cost ← Total_Cost
    Best_Order ← Order
END IF

END FOR

RETURN Best_Order, Minimum_Cost

END FUNCTION

Best_Order, Minimum_Cost ← Find_Best_Join_Order(Relations)

DISPLAY "Optimal Join Order:", Best_Order
DISPLAY "Estimated I/O Cost:", Minimum_Cost

END

```

Explanation:

The algorithm generates possible join orders and estimates the disk I/O cost for each one. For every candidate order, the cost of performing successive joins is calculated. The algorithm compares all estimated costs and selects the join order with the minimum I/O cost. This reduces disk access and improves query execution performance.

Q3. Write pseudocode for the Nested Loop Join algorithm and analyze its time complexity in terms of buffer pages and tuple counts.

Answer:

Algorithm: Nested Loop Join

BEGIN

INPUT Relation R

INPUT Relation S

INPUT Join_Condition

Result $\leftarrow$ EMPTY

FOR each Tuple_R IN R DO

 FOR each Tuple_S IN S DO

 IF Join_Condition(Tuple_R, Tuple_S) = TRUE THEN

 Joined_Tuple $\leftarrow$ CONCATENATE(Tuple_R, Tuple_S)

 ADD Joined_Tuple TO Result

 END IF

 END FOR

END FOR

DISPLAY Result

END

Tuple-Based Time Complexity:

Let:

n = number of tuples in Relation R

m = number of tuples in Relation S

For each tuple in R, all tuples in S are examined.

Therefore:

Time Complexity = $O(n \times m)$

Block Nested Loop Join Using Buffer Pages:

Let:

BR = number of pages occupied by R

BS = number of pages occupied by S

M = number of available buffer pages

The outer relation can use approximately:

M - 2 buffer pages

Hence the I/O cost is:

Cost = BR + CEIL(BR / (M - 2)) $\times$ BS

Explanation:

Nested Loop Join compares every tuple of the outer relation with every tuple of the inner relation. Therefore, its tuple-level complexity is $O(n \times m)$. When buffer pages are used efficiently through Block Nested Loop Join, several pages of the outer relation can be processed together, significantly reducing disk I/O operations.

Q4. Write a pseudocode algorithm to detect deadlocks in a transaction system using a Wait-For Graph cycle detection.

Answer:

Algorithm: Deadlock Detection Using Wait-For Graph

BEGIN

INPUT Active_Transactions

INPUT Lock_Wait_Information

CREATE empty Wait_For_Graph

// Construct Wait-For Graph

FOR each Waiting_Transaction T1 DO

 FOR each Transaction T2 holding required resource DO

 ADD directed edge $T1 \rightarrow T2$
 TO Wait_For_Graph

 END FOR

END FOR

FUNCTION DFS(Transaction)

 State[Transaction] $\leftarrow$ "VISITING"

 FOR each Neighbor OF Transaction DO

 IF State[Neighbor] = "VISITING" THEN
 RETURN TRUE
 END IF

 IF State[Neighbor] = "UNVISITED" THEN

 IF DFS(Neighbor) = TRUE THEN
 RETURN TRUE
 END IF

 END IF

 END FOR

 State[Transaction] $\leftarrow$ "VISITED"

 RETURN FALSE

END FUNCTION

```

FOR each Transaction T DO
    State[T] ← "UNVISITED"
END FOR

Deadlock_Found ← FALSE

FOR each Transaction T DO

    IF State[T] = "UNVISITED" THEN

        IF DFS(T) = TRUE THEN
            Deadlock_Found ← TRUE
            BREAK
        END IF

    END IF

END FOR

IF Deadlock_Found = TRUE THEN

    DISPLAY "Deadlock Detected"

    Victim ← SELECT_TRANSACTION_FOR_ROLLBACK()

    ROLLBACK Victim

    RELEASE all locks held by Victim

ELSE

    DISPLAY "No Deadlock Detected"

END IF

END

```

Explanation:

A Wait-For Graph represents transactions as vertices. An edge $T1 \rightarrow T2$ indicates that transaction $T1$ is waiting for a resource held by $T2$. Depth First Search is used to detect a cycle. If a cycle exists, a deadlock is present. The DBMS selects one transaction as a victim, rolls it back, and releases its locks to break the deadlock.

Q5. Design a pseudocode for the Basic Two-Phase Locking protocol including lock request, grant, wait, and release phases.

Answer:

Algorithm: Basic Two-Phase Locking Protocol

BEGIN

FOR each Transaction T DO

 Phase[T] ← "GROWING"

 Held_Locks[T] ← EMPTY

END FOR

FUNCTION Request_Lock(T, Data_Item, Lock_Type)

 IF Phase[T] = "SHRINKING" THEN

 DISPLAY "New lock cannot be acquired in shrinking phase"

 ABORT T

 RETURN

 END IF

 IF Lock_Is_Compatible(Data_Item, Lock_Type) = TRUE THEN

 GRANT Lock_Type on Data_Item TO T

 ADD Data_Item TO Held_Locks[T]

 DISPLAY "Lock Granted"

 ELSE

 ADD T TO Waiting_Queue(Data_Item)

 DISPLAY "Transaction Waiting"

 WAIT until lock becomes available

 Request_Lock(T, Data_Item, Lock_Type)

 END IF

END FUNCTION

FUNCTION Release_Lock(T, Data_Item)

```

IF Data_Item IN Held_Locks[T] THEN

    Phase[T] ← "SHRINKING"

    RELEASE lock on Data_Item

    REMOVE Data_Item FROM Held_Locks[T]

    WAKE_UP waiting transactions

END IF

END FUNCTION

FUNCTION Commit_Transaction(T)

    FOR each Lock IN Held_Locks[T] DO

        RELEASE Lock

    END FOR

    Held_Locks[T] ← EMPTY

    DISPLAY "Transaction Committed"

END FUNCTION

END

```

Explanation:

Basic Two-Phase Locking contains two phases:

Growing Phase: A transaction can acquire new locks but cannot release locks.

Shrinking Phase: Once a transaction releases its first lock, it enters the shrinking phase. It can release locks but cannot obtain new locks.

This protocol guarantees conflict serializability and helps maintain database consistency.

Q6. Write pseudocode for timestamp-based concurrency control using Thomas' Write Rule for handling conflicting read/write operations.

Answer:

Algorithm: Timestamp Ordering with Thomas' Write Rule

BEGIN

FOR each Transaction T DO

$TS[T] \leftarrow \text{UNIQUE_TIMESTAMP}()$

END FOR

FOR each Data_Item X DO

$\text{Read_TS}[X] \leftarrow 0$

$\text{Write_TS}[X] \leftarrow 0$

END FOR

FUNCTION READ(T, X)

 IF $TS[T] < \text{Write_TS}[X]$ THEN

 DISPLAY "Read Conflict"

 ABORT T

 RESTART T with new timestamp

 ELSE

 READ value of X

$\text{Read_TS}[X] \leftarrow \text{MAX}(\text{Read_TS}[X], TS[T])$

 DISPLAY "Read Successful"

 END IF

END FUNCTION

FUNCTION WRITE(T, X, New_Value)

 IF $TS[T] < \text{Read_TS}[X]$ THEN

 DISPLAY "Write Conflict"

 ABORT T

 RESTART T with new timestamp

 ELSE IF $TS[T] < \text{Write_TS}[X]$ THEN

 // Thomas' Write Rule

 DISPLAY "Obsolete Write Ignored"

```

// Do not abort transaction
RETURN

ELSE

    X ← New_Value

    Write_TS[X] ← TS[T]

    DISPLAY "Write Successful"

END IF

END FUNCTION

END

```

Explanation:

Each transaction receives a unique timestamp. Every database item maintains a read timestamp and a write timestamp.

If an old transaction tries to read a value already written by a newer transaction, it is aborted.

If an old transaction attempts to write a value that has already been read by a newer transaction, it is also aborted.

Under Thomas' Write Rule, when a transaction attempts an outdated write where:

$TS(T) < Write_TS(X)$

the write operation is simply ignored instead of aborting the transaction. This improves concurrency by avoiding unnecessary rollbacks.

Q7. Write a pseudocode algorithm for the ARIES recovery technique with Analysis, Redo, and Undo phases.

Answer:

Algorithm: ARIES Recovery

BEGIN

INPUT Transaction_Log

INPUT Last_Checkpoint

// -----

// PHASE 1: ANALYSIS

// -----

Transaction_Table ← EMPTY

Dirty_Page_Table ← EMPTY

Log_Record ← Last_Checkpoint

WHILE Log_Record exists DO

IF Log_Record represents transaction activity THEN

UPDATE Transaction_Table

END IF

IF Log_Record modifies a page AND

page NOT IN Dirty_Page_Table THEN

ADD page and its LSN

TO Dirty_Page_Table

END IF

IF Log_Record = COMMIT THEN

MARK transaction as COMMITTED

END IF

IF Log_Record = ABORT THEN

MARK transaction as ABORTING

END IF

MOVE to next Log_Record

END WHILE

// -----

// PHASE 2: REDO

// -----

```

Start_LSN ← MINIMUM_REC_LSN(Dirty_Page_Table)

FOR each Log_Record FROM Start_LSN TO END_OF_LOG DO

    IF Log_Record represents database modification THEN

        Page ← Log_Record.Page

        IF Page IN Dirty_Page_Table AND
            Log_Record.LSN >= Dirty_Page_Table[Page].RecLSN AND
            PageLSN(Page) < Log_Record.LSN THEN

            REDO Log_Record

            PageLSN(Page) ← Log_Record.LSN

        END IF

    END IF

END FOR

```

```

// -----
// PHASE 3: UNDO
// -----

```

```

Loser_Transactions ←
    FIND_UNCOMMITTED_TRANSACTIONS(Transaction_Table)

FOR each Transaction T IN Loser_Transactions DO

    Current_LSN ← LastLSN[T]

    WHILE Current_LSN exists DO

        Log_Record ← GET_LOG(Current_LSN)

        IF Log_Record is undoable THEN

            UNDO Log_Record

            WRITE Compensation_Log_Record

        END IF

        Current_LSN ← Log_Record.PrevLSN

    END WHILE

END FOR

```

WRITE END record for T

END FOR

DISPLAY "Database Recovery Completed"

END

Explanation:

ARIES recovery consists of three phases.

Analysis Phase: Identifies active transactions and dirty pages at the time of failure.

Redo Phase: Repeats necessary updates to ensure all committed changes are reflected in the database.

Undo Phase: Reverses changes made by incomplete transactions and creates Compensation Log Records (CLRs).

ARIES therefore follows the principle of repeating history during Redo and removing incomplete work during Undo.

Q8. Design pseudocode for Shadow Paging recovery: maintain current and shadow page tables and roll back on failure.

Answer:

Algorithm: Shadow Paging Recovery

BEGIN

// Transaction Initialization

Shadow_Page_Table ← COPY(Current_Page_Table)

Working_Page_Table ← COPY(Current_Page_Table)

Transaction_Status ← "ACTIVE"

FUNCTION WRITE_PAGE(Page_Number, New_Data)

 New_Page ← ALLOCATE_NEW_DISK_PAGE()

 WRITE New_Data TO New_Page

 Working_Page_Table[Page_Number] ← New_Page

END FUNCTION

FUNCTION COMMIT_TRANSACTION()

 FLUSH all modified pages TO DISK

 FLUSH Working_Page_Table TO DISK

 Current_Page_Table ← Working_Page_Table

 UPDATE database pointer TO Current_Page_Table

 DELETE obsolete shadow pages if safe

 Transaction_Status ← "COMMITTED"

 DISPLAY "Transaction Committed"

END FUNCTION

FUNCTION SYSTEM_FAILURE()

 IF Transaction_Status ≠ "COMMITTED" THEN

```
Current_Page_Table ← Shadow_Page_Table

DISCARD Working_Page_Table

DISCARD newly allocated pages

DISPLAY "Transaction Rolled Back"

END IF

END FUNCTION

END
```

Explanation:

In Shadow Paging, the original page table is maintained as the shadow page table and remains unchanged during a transaction. Modified pages are written to new disk locations, and their addresses are stored in a working page table.

If the transaction commits successfully, the database pointer is switched to the new page table. If a failure occurs before commit, the working page table is discarded and the shadow page table is used, automatically restoring the previous database state.

Q9. Write a pseudocode for Immediate Update recovery using UNDO/REDO log processing after a system crash.

Answer:

Algorithm: Immediate Update Recovery Using UNDO/REDO

BEGIN

INPUT Log_File

Committed_Transactions ← EMPTY

Uncommitted_Transactions ← EMPTY

// Step 1: Analyze Log

FOR each Log_Record IN Log_File DO

IF Log_Record = <START T> THEN

ADD T TO Uncommitted_Transactions

ELSE IF Log_Record = <COMMIT T> THEN

ADD T TO Committed_Transactions

REMOVE T FROM Uncommitted_Transactions

END IF

END FOR

// Step 2: REDO committed transactions

FOR each Log_Record FROM START TO END DO

IF Log_Record is

<T, Data_Item, Old_Value, New_Value> THEN

IF T IN Committed_Transactions THEN

Data_Item ← New_Value

DISPLAY "REDO", T, Data_Item

END IF

END IF

END FOR

```

// Step 3: UNDO uncommitted transactions

FOR each Log_Record FROM END TO START DO

    IF Log_Record is
        <T, Data_Item, Old_Value, New_Value> THEN

        IF T IN Uncommitted_Transactions THEN

            Data_Item ← Old_Value

            DISPLAY "UNDO", T, Data_Item

        END IF

    END IF

END FOR

FLUSH database changes TO DISK

DISPLAY "Recovery Completed"

END

```

Explanation:

In Immediate Update, database pages may be modified before the transaction commits.

Therefore, after a crash, both UNDO and REDO operations may be required.

Committed transactions are REDO operations because their updates must exist permanently in the database. Incomplete transactions are UNDO operations because their partially executed updates must be removed.

Hence:

Committed Transaction → REDO

Uncommitted Transaction → UNDO

This maintains atomicity and durability.

Q10. Write pseudocode for a database connectivity manager that handles connection pooling, query execution, and exception rollback.

Answer:

Algorithm: Database Connectivity Manager

BEGIN

CONSTANT MAX_CONNECTIONS $\leftarrow$ 10

Connection_Pool $\leftarrow$ EMPTY

FUNCTION Initialize_Pool()

FOR $i \leftarrow 1$ TO MAX_CONNECTIONS DO

 Connection $\leftarrow$ CREATE_DATABASE_CONNECTION()

 IF Connection is successful THEN

 ADD Connection TO Connection_Pool

 END IF

END FOR

END FUNCTION

FUNCTION Get_Connection()

 WHILE Connection_Pool is EMPTY DO

 WAIT until connection becomes available

 END WHILE

 Connection $\leftarrow$ REMOVE one connection

 FROM Connection_Pool

 RETURN Connection

END FUNCTION

FUNCTION Release_Connection(Connection)

 IF Connection is valid THEN

 ADD Connection TO Connection_Pool

 ELSE

 New_Connection $\leftarrow$ CREATE_DATABASE_CONNECTION()

 IF New_Connection is successful THEN

 ADD New_Connection TO Connection_Pool

END IF
END IF

END FUNCTION

FUNCTION Execute_Query(SQL_Query)

Connection ← Get_Connection()

TRY

BEGIN TRANSACTION using Connection

Result ← EXECUTE SQL_Query
using Connection

COMMIT Transaction

RETURN Result

CATCH Database_Exception

ROLLBACK Transaction

DISPLAY "Database Error Occurred"

RETURN NULL

CATCH General_Exception

ROLLBACK Transaction

DISPLAY "Unexpected Error Occurred"

RETURN NULL

FINALLY

Release_Connection(Connection)

END TRY

END FUNCTION

CALL Initialize_Pool()

INPUT SQL_Query

```
Result ← Execute_Query(SQL_Query)
```

```
IF Result ≠ NULL THEN
```

```
    DISPLAY Result
```

```
END IF
```

```
END
```

Explanation:

The database connectivity manager maintains a fixed pool of reusable database connections. When a query must be executed, an available connection is taken from the pool. A transaction is started and the query is executed.

If execution succeeds, the transaction is committed. If an exception occurs, the transaction is rolled back to prevent incomplete database modifications. Finally, the connection is returned to the pool so that it can be reused.

This approach improves performance, transaction reliability, resource utilization, and error handling.